

In-search filtering using fedoraRole attributes

Swithun Crowe
cs2@st-andrews.ac.uk

October 31, 2011

1 The need for filtering of results

Our digital archive will host files associated with various research projects. We don't anticipate much demand for access to the contents of the archive. But we would like to allow people associated with the projects access to *their* data, although not data from other projects.

To secure objects in Fedora, we are using XACML policies. These will allow only people with the appropriate `fedoraRoles` to access the objects covered by the policies. These work fine in Fedora.

We are using GSearch as the main way in to Fedora. Having implemented the XACML policies in Fedora, we would like GSearch to only display results to which the current user has access in Fedora. It is dissatisfying and a security risk to show a user results which they can't access.

2 Objects in Fedora

As stated, the primary breakdown of objects in Fedora is by project. Inside each project, objects are classified further by resource type. The three resource types are *data* (research data produced by the project), *source code* (code from programs written for the project), and *documentation* (program documentation, data schemata etc.).

We are ingesting objects into Fedora from Alfresco. A file is ingested into Alfresco, and Alfresco generates a METS document containing the original file and associated metadata datastreams as a mixture of externally referenced and inline XML datastreams. Alfresco generates the PID for Fedora to use, based on a UUID which Alfresco itself uses for identifying the original file. The PID namespace is made up of a code for the project and a code for the reference type, with the UUID as the object ID. For example,

`rps-data:50a1cf1f-747c-4bbb-88b2-911992fdebcb`

3 Search result filtering options

Having thought about the three options for search filtering (post-, pre- and in-search), I decided to go for in-search filtering. Filtering is done by rewriting the query to exclude results which the user can't access in Fedora (rather than collecting search results and then testing the user's access to them using Fedora's XACML policies, or maintaining a different index for each project).

One advantage of post-search filtering is that the same XACML policies control access both in Fedora and GSearch. And with pre-search filtering, the query is shorter and the indexes smaller. But post-search filtering is expensive and slow, and I want to retain the ability to search across several projects.

A stated disadvantage of in-search filtering is that it requires extra index fields. Whether it is a good idea, or not, I found that by using PIDs structured as above, the PID field could be used to filter results, with `fedoraRoles` mapped to particular projects and resource types.

The `fedoraRole` values are composed of a project code, an underscore, and a project role (such as `admin` or `reader`). A project reader, for example, can see the data from the project, but not objects of other resource types. A project admin can see all objects for a project.

As per the documentation, GSearch needs to be configured, so that it knows which kind of search filtering you want to do, and where to find the class to do it.

Listing 1: `fedoragsearch.properties`

```
fedoragsearch.searchResultFilteringModule = \\
    dk.defxws.fedoragsearch.server.SearchResultFilteringDemoImpl
fedoragsearch.searchResultFilteringType = insearch
```

4 Extending GSearch

The servlet filters which GSearch uses for enforcing authentication (using an XML users' file or LDAP) inject `fedoraRole` values into an attribute of the `HttpServletRequest` object which gets passed into `RESTImpl`. This method in `RESTImpl` returns any `fedoraRole` values in an array of `Strings`.

Listing 2: `RESTImpl.getFedoraRoles`

```
/** getFedoraRoles
 * get array of Strings containing fedoraRole values from request attribute
 * @param request HttpServletRequest containing the attribute to look for
 * @return String[] containing the fedoraRole values, or null on error
 */
private String[] getFedoraRoles(HttpServletRequest request)
{
```

```

String fedoraRoles[] = null;

do
{
    Map<String, Set<String>> attributes =
        (Map) request.getAttribute("FEDORA_AUX_SUBJECT_ATTRIBUTES");

    // no FEDORA_AUX_SUBJECT_ATTRIBUTES attribute in request object
    if (null == attributes)
    {
        break;
    }

    Set<String> roles = attributes.get("fedoraRole");

    // no fedoraRole given
    if (null == roles || 0 == roles.size())
    {
        break;
    }

    fedoraRoles = roles.toArray(new String[roles.size()]);
}
while (false);

return fedoraRoles;
}

```

The fedoraRoles array can be passed to the Config object which creates the OperationsImpl object.

Listing 3: RESTImpl.gfindObjects

```

private String gfindObjects(HttpServletRequest request,
    HttpServletResponse response) throws java.rmi.RemoteException
{
    . . .
    String[] fedoraRoles = getFedoraRoles(request);
    Operations ops = config.getOperationsImpl(request.getRemoteUser(),
        indexName, fedoraRoles);
    . . .
}

```

In getOperationsImpl, Config passes the fedoraRoles array to the OperationsImpl object, when initialising it.

Listing 4: Config.getOperationsImpl

```

public GenericOperationsImpl getOperationsImpl(String fgsUserNameParam,
    String indexNameParam, String[] fedoraRoles) throws ConfigException
{
    . . .
    ops.init(fgsUserNameParam, indexName, this, fedoraRoles);
    return ops;
}

```

In `GenericOperationsImpl`'s `init` method, the `fedoraRoles` array is stored as a protected class member, so that it can be accessed by the subclass `OperationsImpl`.

Listing 5: `GenericOperationsImpl.init`

```
. . .
protected String[] fedoraRoles;
. . .
public void init(String fgsUserName, String indexName, Config currentConfig,
    String[] fedoraRoles)
{
    . . .
    this.fedoraRoles = fedoraRoles;
    . . .
}
```

`RESTImpl` calls the `OperationsImpl` `gfindObjects` method. In this method, if *insearch* filtering is desired, the `fedoraRoles` array and the `Config` object are passed to the method for rewriting the query for *insearch* filtering.

Listing 6: `OperationsImpl.gfindObjects`

```
public String gfindObjects(. . .
{
    . . .
    String usingQuery = query;
    if (srf != null && config.isSearchResultFilteringActive("insearch"))
    {
        usingQuery = srf.rewriteQueryForInsearch(fgsUserName,
            usingIndexName, query, fedoraRoles, config);
        . . .
    }
    . . .
}
```

In the class which implements *insearch* filtering, the `fedoraRoles` array is used to generate the rewritten query. If the array is null, then this means that the user has no `fedoraRole`, so shouldn't be allowed to see anything. The original query is made to negate itself, so that nothing is found.

If the user has administrator as one of their roles (e.g. if they are `fedoraAdmin`), then they are allowed to see everything (that they search for). In this case, the rewritten query is the same as the original query, as no filtering is needed.

If a `fedoraRole` contains an underscore, then it is split on the first underscore. The first element is taken to be the project code, and the second, the project role. These are used to obtain a string which can be formatted to produce a query clause for filtering out search results that the user shouldn't see.

Listing 7: SearchResultFilteringDemoImpl.rewriteQueryForInsearch

```
public String rewriteQueryForInsearch(String fgsUserName, String indexName,
String query, String[] fedoraRoles, Config config)
throws java.rmi.RemoteException
{
    String rewrittenQuery = null;

    do
    {
        boolean isAdmin = false;
        String clause = "";

        // use roles to create disjointed clause for filter
        for (String role : fedoraRoles)
        {
            // admin role trumps everything else
            if (role.equals("administrator"))
            {
                isAdmin = true;
                break;
            }

            // need to split on first _ and need config to be not null
            if (-1 == role.indexOf('_') || null == config)
            {
                continue;
            }

            String[] parts = role.split("_", 2);

            // look for specific/fallback filter
            String filter = config.getFilterProperty(parts[0] + "." + parts[1]);

            // no filter, so go to next role
            if (filter.equals(""))
            {
                continue;
            }

            // not first role, so need OR before this disjunct
            if (!clause.equals(""))
            {
                clause += "_OR_";
            }

            // use filter to add to clause
            clause += String.format(filter, parts[0]);
        }

        // if admin, then no need to rewrite query
        if (isAdmin)
        {
            rewrittenQuery = query;
        }
    }
}
```

```

        break;
    }

    // no clause generated, so can't get anything
    if (clause.equals(""))
    {
        rewrittenQuery = "(" + query + ")_AND_NOT_(" + query + ")";
        break;
    }

    // add clause to query
    rewrittenQuery = "(" + query + ")_AND_(" + clause + ")";
}
while (false);

return rewrittenQuery;
}

```

Config's method `getFilterProperty` takes project codes and project roles, and looks for either a property which is specific to both the project and the role, or a fallback property, which is specific to just the role.

Listing 8: Config.getFilterProperty

```

/** getFilterProperty
 * return a string that can be used to filter a query
 * using the supplied project and resource type codes
 * @param project String holding project code
 * @param projectRole String holding project role
 * @return String containing matched property value, or empty if not found
 */
public String getFilterProperty(String project, String projectRole)
{
    String filter = "";
    do
    {
        // prefix of properties
        String filterPrefix = "fedoragsearch.filter.";

        // look for project specific filter
        filter = filterProps.getProperty(filterPrefix + project +
            "." + projectRole);
        if (null != filter)
        {
            break;
        }

        // look for fallback filter
        filter = filterProps.getProperty(filterPrefix + "fallback." +
            projectRole);
        if (null != filter)
        {
            break;
        }
    }
}

```

```

        // nothing found, so make filter empty string
        filter = "";
    }
    while (false);

    return filter;
}

```

The properties are kept in a file, `filter.properties`. An example, containing a property specific to **readers** of the **rps** project, and a property for **admins** of all projects is:

```

admin query for all projects fedoragsearch.filter.fallback.admin=PID:%s-
*

```

The placeholder (%s) will be replaced by the project code.

The properties file is loaded by `Config`'s constructor method, and the properties are kept in a private class member.

Listing 9: `Config.Config`

```

private Properties filterProps = null;
...
public Config(String configNameIn) throws ConfigException
{
    ...
    // Get search result filter properties
    try
    {
        do
        {
            InputStream propStream = Config.class.getResourceAsStream("/" +
                configName + "/filter.properties");
            // properties not necessary
            if (propStream == null)
            {
                if (logger.isDebugEnabled())
                {
                    logger.debug(configName +
                        "/filter.properties_not_found_in_classpath");
                }
                break;
            }

            filterProps = new Properties();
            filterProps.load(propStream);
            propStream.close();

            if (logger.isInfoEnabled())
            {
                logger.info("filter.properties=" + filterProps.toString());
            }
        }
        while (false);
    }
}

```

```

    }
    catch (IOException e)
    {
        throw new ConfigException("***_Error_loading_" + configName +
            "/filter.properties:\n" + e.toString());
    }
    . . .
}

```

5 Result

With these modifications, it is possible to access the `fedoraRole` values which get set when the user authenticates, and then to use these to filter search results by rewriting the query used to obtain the search results.

Using a properties file means that new projects can be added and project specific filters written without touching the Java code.